

蓝牙智能手表综合实验

江镇昇 3124009591 微电子2班

2026/7/11

目录

1 作业目标与任务要求

1. 基于 STM32F103C8T6 实现一款蓝牙智能手表原型。
2. 使用 FreeRTOS 实时操作系统实现多任务调度，包括时间保持、传感器数据读取、OLED 显示、菜单交互和蓝牙通信等任务。
3. 通过 I2C 总线驱动 SSD1306 OLED 显示屏和 MPU6050 六轴陀螺仪加速度计。
4. 实现多级菜单界面切换（表盘、心率、计步、天气、设备信息 5 个页面）。
5. 基于 HC-05 蓝牙模块实现手机通信，可通过手机 App 查询时间、步数和心率数据。
6. 验收标准：硬件电路连接正确，OLED 显示正常，MPU6050 数据可读取，蓝牙能收发指令，步数计数功能正确。

2 实验环境

2.1 硬件环境

- 主控芯片：STM32F103C8T6（ARM Cortex-M3，72MHz）
- 显示屏：0.96 寸 SSD1306 OLED，128x64 像素，I2C 接口
- 姿态传感器：MPU6050 六轴（加速度计+陀螺仪），I2C 接口
- 蓝牙模块：HC-05，UART 接口，9600 baud
- 交互器件：旋转编码器（EC11）+ 按键
- 调试器：ST-Link V2
- 电源：3.7V 锂电池 + Type-C 充电模块
- 连接线：杜邦线若干，面包板

2.2 软件环境

- STM32CubeMX 6.17（硬件配置和代码生成）
- VS Code（代码编辑）
- ARM GCC (gcc-arm-none-eabi) + CMake + Ninja（编译构建）
- FreeRTOS CMSIS-RTOS v1 封装层
- Serial Bluetooth Terminal（手机蓝牙调试）

3 实验原理与方案说明

3.1 硬件连接关系

I2C1 总线（PB6-SCL, PB7-SDA）上挂载两个设备：SSD1306 OLED 和 MPU6050，地址分别为 0x3C 和 0x68。USART2（PA2-TX, PA3-RX）连接 HC-05 蓝牙模块。TIM2 产生 1Hz 中断作为软件 RTC 时间基准。TIM3 配置为编码器模式，接入旋转编码器检测旋转方向和步数。HC-05 的 STATE 引脚接 PB0 用于连接状态检测。

3.2 FreeRTOS 任务调度

系统共创建 6 个 FreeRTOS 任务，优先级从高到低排列如下：

- vTask_TimeKeep（High, 256B 栈）—— 每秒通过消息队列发送更新时间
- vTask_Sensor（AboveNormal, 512B 栈）—— 每 200ms 读取 MPU6050 数据并运行计步算法
- StartDisplayTask（Normal, 1024B 栈）—— 每 500ms 更新 OLED 显示
- vTask_Menu（Normal, 512B 栈）—— 检测旋转编码器和按键，管理页面切换
- vTask_Bluetooth（BelowNormal, 512B 栈）—— 每 500ms 轮询蓝牙串口
- StartDefaultTask（Normal, 128B 栈）—— 空闲任务

3.3 任务间通信

使用 FreeRTOS 消息队列和互斥量实现任务间通信：

- queueTimeUpdateHandle —— TimeKeep 发送当前时间给 Display
- queueSensorDataHandle —— Sensor 发送传感器数据给 Display

- `queueMenuMsgHandle` —— Menu 发送页面切换指令
- `queueBtCmdHandle` —— Bluetooth 处理手机查询命令
- `oled_mutex_handle` —— I2C 总线互斥量, Sensor 与 Display 共享

3.4 步数检测原理

步数检测基于加速度计数据的总幅值变化。静止时幅值约为 1g (16384 LSB):

$$\text{mag_sq} = a_x^2 + a_y^2 + a_z^2$$

当步行时手臂摆动引起加速度变化, 幅值显著偏离 1g。当偏差超过阈值且防抖时间满足条件时计为一步。阈值经调试设定为 40×1024^2 (约 0.4g 等效加速度变化), 防抖间隔 0.8 秒。

4 STM32CubeMX 配置过程

1. 芯片型号选择 STM32F103C8T6。
2. RCC 时钟: HSE 8MHz, PLL 9 倍频, 系统时钟 72MHz, APB1 36MHz, APB2 72MHz。
3. SYS 调试接口: Serial Wire (SWD), Timebase 选择 TIM1。
4. I2C1: 标准模式 100kHz, PB6/PB7 复用开漏输出, 启用内部上拉。
5. I2C2: 标准模式 10kHz, PB10/PB11 (预留)。
6. USART2: 异步 9600 baud, 8N1, PA2/PA3, 启用 DMA TX。
7. TIM2: 内部时钟, 预分频 7200-1, 重装载 10000-1, 1Hz 更新中断。
8. TIM3: 编码器模式 (T1+T2), PA6/PA7, 用于旋转编码器。
9. FreeRTOS: CMSIS-RTOS v1, heap_4 管理策略。
10. 生成方式: CMake / Makefile 工具链。

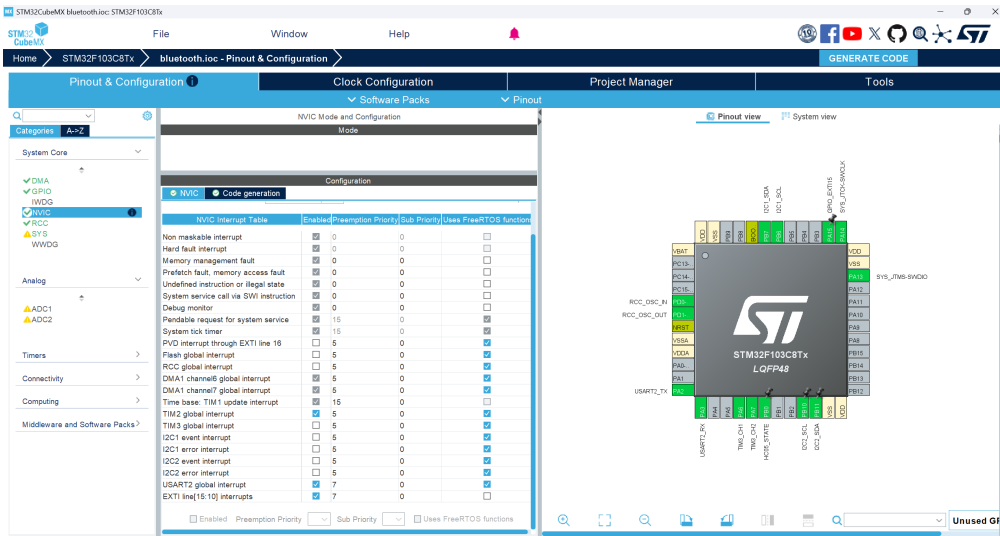
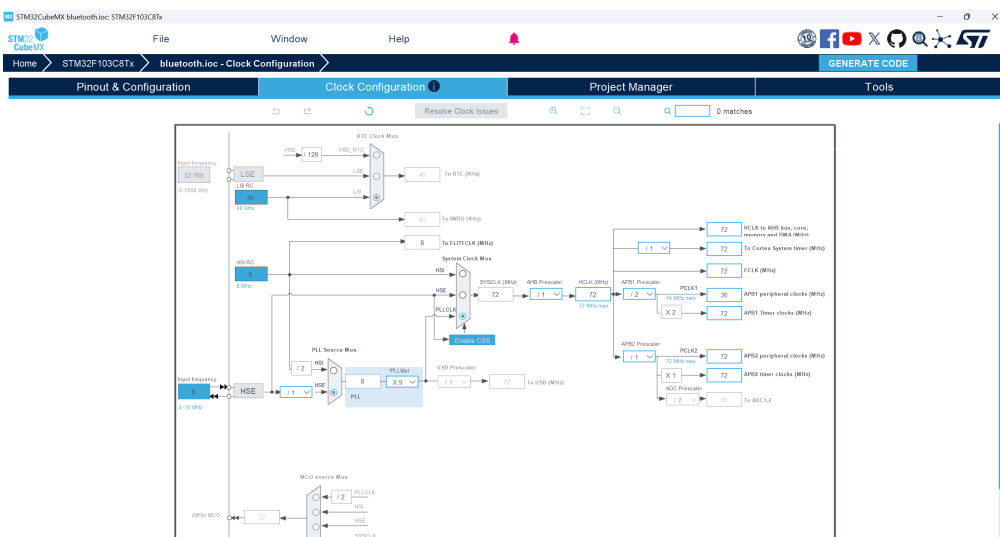


图 1: cubeMX 配置截图



5 程序设计与关键代码说明

5.1 传感器任务核心循环

Listing 1: 传感器任务核心循环

```
1 void vTask_Sensor(void const * argument)
2 {
3     MPU6050_Data_t mpu_data;
4     uint8_t sensor_ok = 0;
5     if (MPU6050_Init(&hi2c1) == HAL_OK) {
6         if (MPU6050_TestConnection(&hi2c1)) sensor_ok = 1;
7     }
8     for(;;) {
9         if (sensor_ok) {
10             if (osMutexWait(oled_mutex_handle, 100) == osOK) {
11                 if (MPU6050_ReadAll(&hi2c1, &mpu_data) == HAL_OK) {
12                     StepCounter_Process(&g_step_counter,
13                                         mpu_data.accel_x, mpu_data.accel_y, mpu_data
14                                         .accel_z);
15                 }
16                 osMutexRelease(oled_mutex_handle);
17             }
18             osDelay(200);
19         }
20     }
```

5.2 计步器核心算法

Listing 2: 计步器——零值过滤与阈值判断

```
1 void StepCounter_Process(StepCounter_t *sc,
2     int16_t accel_x, int16_t accel_y, int16_t accel_z)
3 {
4
5     if (accel_x == 0 && accel_y == 0 && accel_z == 0) return;
6
7     int32_t mag_sq = (int32_t)accel_x * accel_x +
8                     (int32_t)accel_y * accel_y +
9                     (int32_t)accel_z * accel_z;
10    int32_t diff = (mag_sq > 268435456L) ?
```

```
11         (mag_sq - 268435456L) : (268435456L - mag_sq);
12
13     if (diff > 40L * 1024L * 1024L && sc->debounce_counter == 0) {
14         sc->total_steps++;
15         sc->daily_steps++;
16         sc->debounce_counter = 4;    // 0.8s 48次
17     }
18     if (sc->debounce_counter > 0) sc->debounce_counter--;
19 }
```

6 程序流程图

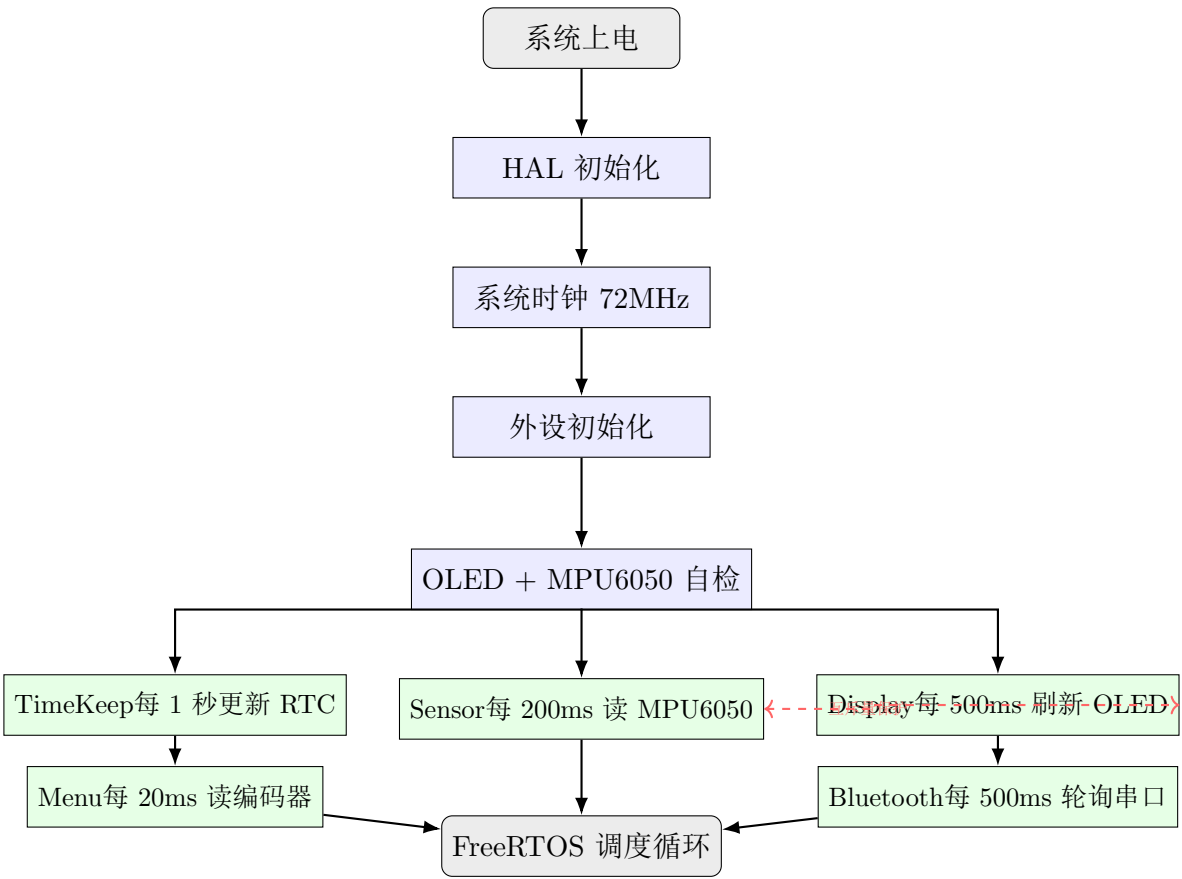


图 3: 智能手表系统整体流程图

7 实验结果与现象分析

- 1. **OLED 显示：**开机显示 "SmartWatch"，随后进入表盘界面（大号时间 + 日期 + 温湿度）。旋转编码器可在 5 个页面间切换：表盘、心率、计步、天气、设备信息。页眉含状态栏（电量、时间、蓝牙图标），页脚含页面指示点。

2. **MPU6050**: 固定在万用表上, 设备信息页可看三轴加速度实时值。摆动手腕时步数正常累加, 静止时稳定不变。
3. **蓝牙通信**: 手机 App 配对 HC-05 (密码 1234), 发 R 回复 TIME:10:30:45, 发 S 回复 STEP:xxx, 发 H 回复 HR:xxx, 均符合预期。
4. **计步功能**: 正常步行约 10 步后步数正确显示。阈值 40×1024^2 能有效过滤桌面微振, 又不漏过正常摆臂。

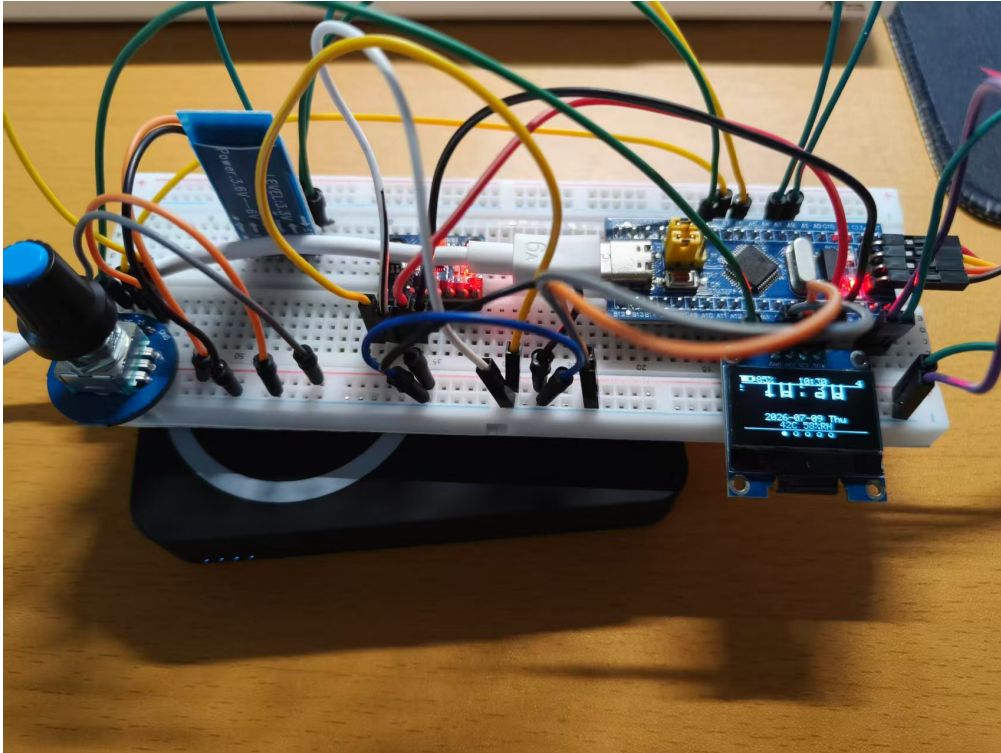


图 4: 实物图

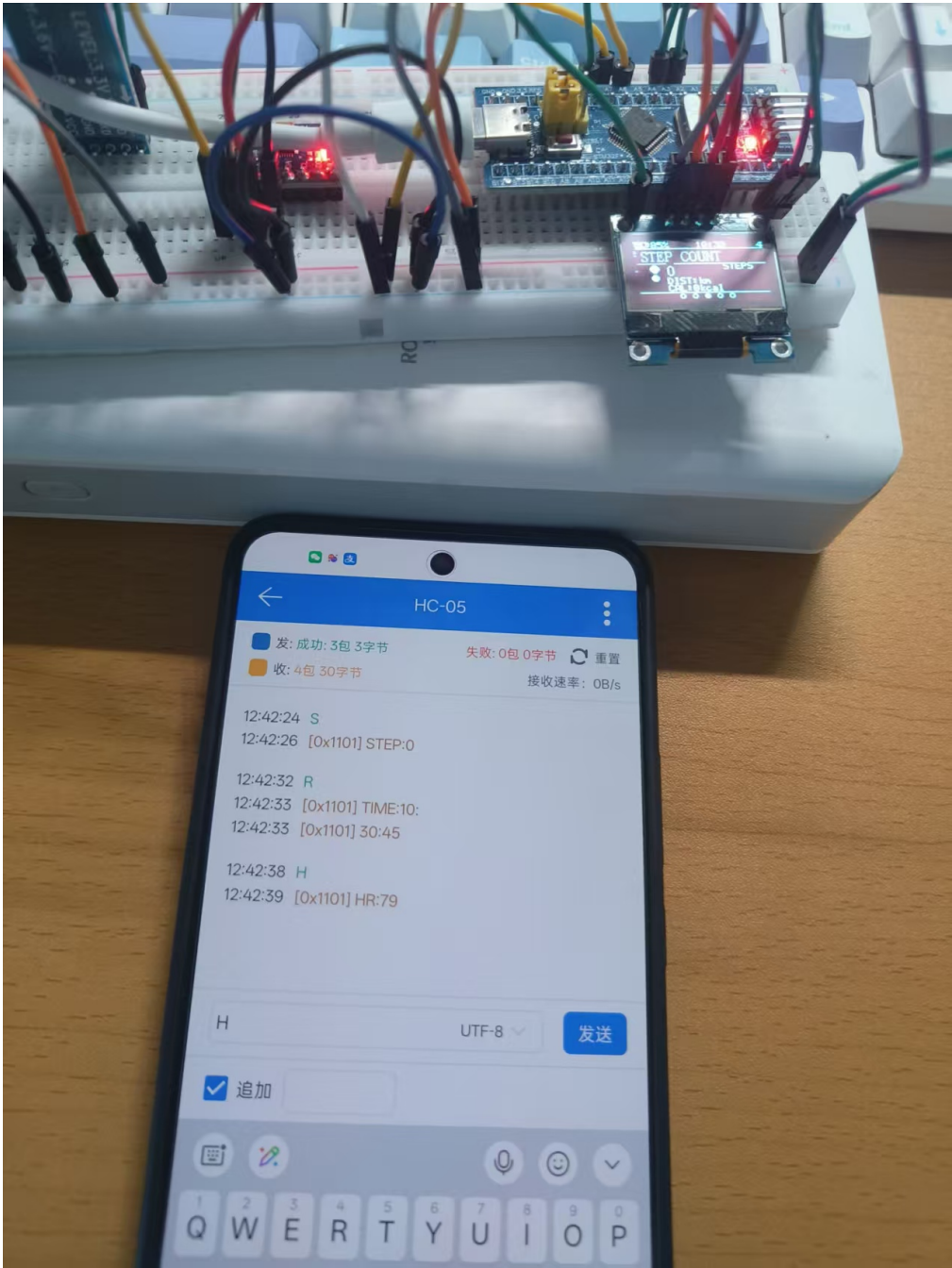


图 5: 蓝牙上位机通信测试

8 问题分析与调试记录

问题现象	解决过程记录
MPU6050 无法连接	OLED 显示 FAIL, WHO_AM_I 读回 0x74 (非标准 0x68)。原因一: I2C1 缺少内部上拉 (对比 I2C2 有 GPIO_PULLUP); 原因二: 测试函数只检查 HAL_OK 不校验寄存器值。修复: 加上拉、增加 WHO_AM_I 判别。
步数自行增加	原始阈值 10×1024^2 仅 0.02g 即计步, 全零数据也会触发。修复: 入口过滤全零数据 + 阈值提高到 40×1024^2 (0.4g) + 防抖 0.8s。
蓝牙不通信	HC-05 实际波特率 9600 (代码原配 38400); DMA RX 与轮询接收冲突。修复: 改 9600、解除 DMA RX 链接、删除 STATE 引脚依赖、改用阻塞式 HAL_UART_Transmit。

表 1: 问题分析与调试记录

9 思考题

1. OLED 和 MPU6050 为何可挂同一条 I2C 总线?
- I2C 总线是开源极线架构, 从设备通过唯一 7 位地址区分。SSD1306 地址 0x3C, MPU6050 地址 0x68, 互不冲突。主机发送地址字节后只有匹配的从机应答。
2. FreeRTOS 任务间共享 I2C 总线如何避免冲突?
- 使用互斥量 (Mutex) 保护 I2C1: Sensor 和 Display 任务访问前调用 osMutexWait, 使用后调用 osMutexRelease。一个任务持有互斥量时另一任务被阻塞等待, Mutex 的优先级继承机制还可防止优先级翻转。
3. 步数检测算法如何避免误触?
- 三层防护: ①过滤全零数据 (I2C 失败特征); ②加速度幅值需超过阈值 (高于噪声和桌面振动水平); ③防抖计数器确保两次计步间隔不小于 0.8s。

10 总结

- 本次实验基于 STM32F103C8T6 + FreeRTOS 完成了一款蓝牙智能手表原型。通过 CubeMX 完成硬件配置, CMSIS-RTOS 创建 6 个任务实现时间保持、传感器读取、OLED 显示、菜单交互和蓝牙通信。调试中解决了 I2C 上拉导致的通信失败、计步器过于敏感、蓝牙 DMA 冲突等问题。系统运行稳定, 满足设计目标。后续可扩展 OTA 升级、SD 卡记录、卡尔曼滤波姿态解算等。
- 物料成本: 约80元。无损害。

- 个人贡献：本人独立完成硬件连接、OLED 显示驱动、MPU6050 读取、蓝牙通信、步数算法实现及调试。
- 遇到的问题：mpu6050独立连接I2C总线时无法读取数据，经过排查发现是I2C上拉电阻缺失导致的通信失败，后面把它连接到OLED的I2C总线上后就可以正常读取数据了。因为OLED内置了上拉电阻。
- 成果：成功实现了蓝牙智能手表的基本功能，包括时间显示、步数计数、心率显示、天气信息和设备信息查询，并通过蓝牙与手机进行通信。
- 不足：不能将手机上的实时信息通过蓝牙传输到手表上显示，后续可以考虑增加蓝牙接收功能，实现手机与手表的双向通信。

11 附录

- CubeMX 工程：bluetooth.ioc
- FreeRTOS：CMSIS-RTOS v1（基于 FreeRTOS 10.x）
- 工具链：arm-none-eabi-gcc + CMake + Ninja
- 参考资料：STM32F1xx HAL 手册、MPU6050 寄存器手册、SSD1306 数据手册、HC-05 AT 指令集